

The Devil Within, The Cure Without: Securing Locally Private Graph Learning under Poisoning

Longzhu He

Beijing University of Posts and Telecommunications
Beijing, China
helongzhu@bupt.edu.cn

Li Sun*

Beijing University of Posts and Telecommunications
Beijing, China
lsun@bupt.edu.cn

Peng Tang

Shandong University
Qingdao, China
tangpeng@sdu.edu.cn

Sen Su*

Beijing University of Posts and Telecommunications
Beijing, China
Chongqing University of Posts and Telecommunications
Chongqing, China
susen@bupt.edu.cn

Abstract

Graph neural networks (GNNs) have become the cornerstone of graph representation learning in real-world applications, especially in social networks such as Facebook and Twitter, where user interactions naturally form graph-structured data. To safeguard sensitive information, including user attributes and personal connections, locally private graph learning protocols employ local differential privacy (LDP) to provide rigorous user-level guarantees. Despite these protections, we show that such protocols remain highly vulnerable to data poisoning attacks. Our approach targets the full-privacy setting, where both node features and edges are LDP-protected, and executes coordinated manipulations that significantly degrade utility, such as node classification accuracy, across multiple social network benchmarks. To counter these threats, we propose CURENET, a defense framework with four key components: ① local data perturbation for privacy, ② trimmed screening to filter abnormal submissions, ③ privacy-aware fake node pruning to remove sophisticated adversaries, and ④ a utility enhancement module to recover graph learning performance under privacy constraints. Extensive experiments on four real-world social network datasets—LastFM, Twitch, Github, and Facebook—demonstrate both the severe impact of our attack and the effectiveness of our defense. This study underscores the critical need to secure privacy-preserving graph learning against sophisticated poisoning threats.

CCS Concepts

• Security and privacy → Social network security and privacy.

Keywords

Social Network, Graph Neural Network, Privacy-Preserving, Data Poisoning Attack

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792102>

ACM Reference Format:

Longzhu He, Peng Tang, Li Sun, and Sen Su. 2026. The Devil Within, The Cure Without: Securing Locally Private Graph Learning under Poisoning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792102>

1 Introduction

Graph Neural Networks (GNNs) [69] have emerged as a fundamental paradigm for learning from graph-structured data, demonstrating powerful representation capabilities across a broad spectrum of applications, such as social sciences [32, 57], graph mining [18, 27], and recommendation systems [67, 72]. Despite their success, deploying GNNs to real-world networks often entails handling sensitive personal or behavioral information, where even indirect access to trained models may lead to privacy leakage [41, 53, 63–65, 75]. These risks have sparked increasing concern over data confidentiality, highlighting the urgent need for graph learning frameworks that can preserve individual privacy while maintaining high utility.

A promising direction to addressing this challenge is *locally private graph learning* [15, 16, 20, 34, 38, 45, 47, 51, 77], which adapts the principles of local differential privacy (LDP) [71] to decentralized graph data. As illustrated in Fig. 1(a), each user locally perturbs their sensitive data before transmitting it to an untrusted server, which subsequently performs graph learning on the collected noisy data. This paradigm inherently guarantees user-level privacy while exploiting the noise-resilient nature of GNN message passing [1, 12] to mitigate performance degradation. Consequently, locally private graph learning has emerged as a promising foundation for building secure and trustworthy graph learning systems with high utility.

However, locally private graph learning protocols remain vulnerable to *data poisoning* in realistic distributed environments. An adversary can create sybil accounts [8, 31, 52], craft plausible social identities [54], and send friend requests [39] to genuine users; some recipients, acting in line with normal social behavior [5, 42], may accept these connections, allowing the fake accounts to gain footholds in the network (see Fig. 1(b)). Once embedded, malicious nodes submit carefully crafted, locally perturbed features and adjacency reports to the server. These adversarial signals are then

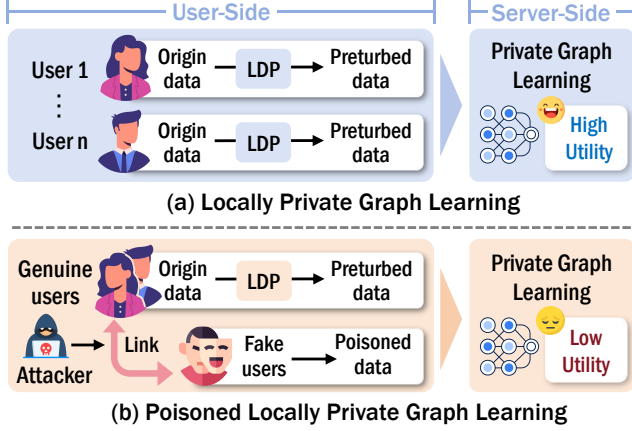


Figure 1: (a) In conventional locally private graph learning, multiple legitimate users perturb their sensitive data using LDP before submitting it to an untrusted server, which then conducts private graph learning. (b) In contrast, our study examines how an attacker can compromise this process: by injecting fake users, establishing malicious links with genuine nodes, and submitting carefully crafted poisoned data to the server, the attacker aims to degrade the learning utility.

amplified by GNN message passing, leading to degraded performance in downstream tasks such as node classification [28, 56] and link prediction [55, 73]. In this work, we systematically investigate the threat of data poisoning under locally private graph learning. Prior studies [16] typically focus on a *partial-privacy* setting, where only node features are perturbed while adjacency lists remain fully observable. In contrast, we examine a more realistic *full-privacy* scenario, in which both features and adjacency lists are protected via LDP. Through empirical evaluation and theoretical analysis, we demonstrate that, data poisoning attack can cause even greater utility degradation under this full-privacy setting, indicating that prior assessments have underestimated the potential impact of adversarial injections. These results highlight that data poisoning continues to pose a serious threat in locally private graph learning, emphasizing the urgent need for effective defense strategies.

Despite the severity of data poisoning in full-privacy graph learning, designing effective defenses remains highly challenging. Several factors contribute to this difficulty. *First*, the distributed nature of local privacy protocols prevents the server from accessing raw node features or adjacency information, limiting the applicability of traditional detection or robust aggregation methods. *Second*, the noise introduced by LDP mechanisms obscures malicious manipulations, reducing the signal available to distinguish poisoned nodes from genuine ones. *Third*, the diversity of LDP mechanisms (e.g., the piecewise mechanism [45, 62], multi-bit mechanism [38, 51], and square wave mechanism [34, 35]) and the variety of GNN architectures (e.g., GCN [28], GraphSAGE [14], and GAT [61]) induce heterogeneous behavior in the perturbed data, complicating the design of universal defenses. Prior studies [15] confirm that conventional GNN poisoning defenses are largely ineffective in such scenarios, highlighting the need for fundamentally new strategies.

To address these challenges, we propose CURENET, a comprehensive defense framework for locally private graph learning under

data poisoning threats. CURENET integrates four collaborative core modules: ① *local data perturbation*, ② *trimmed screening*, ③ *privacy-aware fake node pruning*, and ④ *utility enhancement*. Specifically, CURENET operates in conjunction with diverse LDP mechanisms, leveraging perturbed features, adjacency information, and degree data to robustly detect and mitigate malicious nodes. The trimmed screening and fake node pruning modules filter out suspicious nodes and edges based on statistical and structural cues, while the utility enhancement module optimizes graph representations to reduce the amplification of poisoned signals and preserve downstream performance. We have conducted extensive experiments on multiple real-world social network datasets (LastFM, Twitch, Github, and Facebook [50]) using six different LDP mechanisms and six representative GNN backbones, demonstrating that CURENET consistently mitigates poisoning effects and achieves high utility. Our contributions are summarized as follows:

- ♦ **Comprehensive threat analysis** under a realistic full-privacy setting, revealing that data poisoning attacks inflict substantially greater utility degradation in locally private graph learning.
- ♦ **Unified defense framework CURENET**, consisting of four collaborative core modules that jointly mitigate poisoning effects and preserve high utility in locally private graph learning.
- ♦ **Extensive empirical validation** across six LDP mechanisms, six GNN backbones, and four real-world datasets, demonstrating consistent mitigation of poisoning effects and superior utility.

Organization. The remainder of this paper is organized as follows. Sec. 2 introduces the background knowledge. Sec. 3 revisits poisoning attacks under full-privacy setting, providing both empirical and theoretical insights. Sec. 4 develops the defense framework CURENET, and Sec. 5 reports comprehensive experimental results. Sec. 6 reviews the related literature, and Sec. 7 concludes the paper.

2 Preliminaries

This section introduces the problem definition (Sec. 2.1) and essential background knowledge on Local Differential Privacy (Sec. 2.2) and Locally Private Graph Learning (Sec. 2.3).

2.1 Problem Definition

Consider a graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ denotes the set of nodes with $|\mathcal{V}|$ users in total. The feature matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ associates each node $v \in \mathcal{V}$ with a d -dimensional node feature vector $\mathbf{x}_v \in \mathbb{R}^d$, and the adjacency matrix $\mathbf{A} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ encodes the connectivity, where each row $\mathbf{a}_v \in \{0, 1\}^{|\mathcal{V}|}$ represents the adjacency list of user v . Together, $\mathbf{x}_v$ and $\mathbf{a}_v$ encapsulate sensitive personal attributes and social relations within the network. Unlike prior studies [15, 16, 20, 34, 45, 47, 51, 77] that focus partially on protecting node features or adjacency lists, we consider a *full-privacy setting* where both types of information are treated as sensitive. In addition, each node is assigned a categorical label $y_v \in \{0, 1\}^c$ in one-hot encoding, where c is the number of classes.

Under locally private graph learning, each user perturbs its sensitive data $(\mathbf{x}_v, \mathbf{a}_v)$ via LDP mechanisms [70, 71] before uploading it to the server, which subsequently performs GNN-based training on the perturbed graph. This two-phase protocol (*user-side perturbation* and *server-side learning*) enables collaborative graph learning while preserving privacy. In this paper, we consider two representative downstream tasks: node classification [28] and link

prediction [73]. Our goal is to reveal the vulnerability of locally private graph learning protocols to *data poisoning attacks* and to develop effective defenses. Under our threat model, an adversary can inject multiple fake nodes into $\mathcal{G}$ and manipulate both their features and adjacency lists to connect with genuine nodes, thereby degrading the utility of graph learning (e.g., classification accuracy). A detailed formulation of the threat model is provided in Sec. 3.1.

2.2 Local Differential Privacy

Differential Privacy (DP) [10] has emerged as a *de facto* standard for protecting individual data, offering rigorous guarantees against privacy breaches. In decentralized scenarios, Local Differential Privacy (LDP) [17, 70, 71] has received particular attention, as it enables privacy-preserving data collection without requiring trust in a central curator. Major technology companies including Google [11], Apple [59], and Microsoft [7] have widely deployed LDP in practice. In the LDP setting, each user perturbs their raw data locally through a randomized mechanism $\mathcal{M}$ before sending it to the server. The server then aggregates the perturbed data to conduct statistical analysis or machine learning, without ever accessing the unperturbed sensitive data. This paradigm ensures strong protection even under an untrusted aggregator and is formally defined as follows:

Definition 1 (ϵ -LDP). A randomized local mechanism $\mathcal{M}$ satisfies ϵ -local differential privacy, if for all possible pairs of user's private data x and x' , and for any possible output $y \in \text{Range}(\mathcal{M})$, we have:

$$\Pr[\mathcal{M}(x) = y] \leq e^\epsilon \cdot \Pr[\mathcal{M}(x') = y], \quad (1)$$

where $\epsilon > 0$ is the *privacy budget*, controlling the trade-off between privacy and utility: a smaller (*resp.* larger) ϵ provides stronger (*resp.* weaker) privacy guarantees but leads to lower (*resp.* higher) utility.

In private graph learning, the commonly used notion for protecting a user's adjacency list is ϵ -link LDP [23, 38, 48, 77], which extends the standard LDP definition to link data, as shown below:

Definition 2 (ϵ -link LDP). A randomized mechanism $\mathcal{M}$ satisfies ϵ -link local differential privacy (ϵ -link LDP), if for any user v and for any two possible adjacency lists $\mathbf{a}_v, \mathbf{a}'_v \in \{0, 1\}^{|\mathcal{V}|}$ that only differ by one bit, i.e. $\|\mathbf{a}_v - \mathbf{a}'_v\|_{1,1} = 1$, and for any output $y \in \text{Range}(\mathcal{M})$,

$$\Pr[\mathcal{M}(\mathbf{a}_v) = y] \leq e^\epsilon \cdot \Pr[\mathcal{M}(\mathbf{a}'_v) = y]. \quad (2)$$

This definition ensures that the presence or absence of any edge linked to v cannot be inferred with high confidence, thereby protecting relational privacy in networked data. Moreover, LDP enjoys two fundamental properties that enhance its applicability, as follows:

Theorem 1 (Post-Processing Invariance[9]). If $\mathcal{A} : \mathcal{X} \mapsto \mathcal{Z}$ satisfies ϵ -LDP, then for any (possibly randomized) mapping $\mathcal{F} : \mathcal{Z} \mapsto \mathcal{Z}'$, the composed mechanism $\mathcal{A} \circ \mathcal{F} : \mathcal{X} \mapsto \mathcal{Z}'$ satisfies ϵ -LDP.

Theorem 2 (Sequential Composition [26]). If $\mathcal{M}_i, i \in \{1, 2, \dots, n\}$ are applied sequentially on the same user's data, where $\mathcal{M}_i : \mathcal{X} \mapsto \mathcal{Z}_i$ satisfies ϵ_i -LDP, then their composed mechanism $\mathcal{M} = (\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_n) : \mathcal{X} \mapsto \prod_{i=1}^n \mathcal{Z}_i$ satisfies $(\sum_{i=1}^n \epsilon_i)$ -LDP.

2.3 Locally Private Graph Learning

Next, we introduce the locally private graph learning protocol $\mathbb{P}$ operating under full-privacy settings, which consists of two successive basic phases: ① *user-side perturbation* and ② *server-side learning*.

2.3.1 User-Side Perturbation. In this phase, each user v perturbs both its node features $\mathbf{x}_v \in \mathbb{R}^d$ and adjacency list $\mathbf{a}_v \in \{0, 1\}^{|\mathcal{V}|}$ to satisfy LDP. Formally, $\mathbf{x}'_v \leftarrow \mathcal{M}_x(\mathbf{x}_v, \epsilon_x)$, $\mathbf{a}'_v \leftarrow \mathcal{M}_a(\mathbf{a}_v, \epsilon_a)$, where ϵ_x and ϵ_a denote the privacy budgets allocated to node features and adjacency lists, respectively, with $\epsilon_x + \epsilon_a = \epsilon$. Specifically:

- **Node Feature Perturbation.** The general perturbation procedure can be described in two steps: ① select $m \in \{1, \dots, d\}$ dimensions at random from the d -dimensional feature space; ② apply an ϵ_x/m -LDP noise to each chosen dimension, while setting the remaining $d - m$ dimensions to zero. The primary LDP mechanisms for node feature perturbation include the multi-bit mechanism (MB) [38, 51], the square wave mechanism (SW) [34], and the piecewise mechanism (PM) [45, 62]. Using the one-dimensional PM as an example¹, consider an input value x . The mechanism outputs a perturbed value $x' \in [-B, B]$, where $B = \frac{e^{\epsilon/2} + 1}{e^{\epsilon/2} - 1}$. Specifically, x' is drawn from the following probability density function:

$$\Pr[x' = c|x] = \begin{cases} p, & \text{if } c \in [l(x), r(x)] \\ p/e^\epsilon, & \text{if } c \in [-B, l(x)) \cup (r(x), B] \end{cases}, \quad (3)$$

where $p = \frac{e^\epsilon - e^{\epsilon/2}}{2e^{\epsilon/2} + 2}$, $l(x) = \frac{B+1}{2} \cdot x - \frac{B-1}{2}$, and $r(x) = l(x) + B - 1$.

- **Adjacency List Perturbation.** To protect the adjacency list $\mathbf{a}_v$, the randomized response (RR) [20, 25, 38, 47, 77] is commonly employed to achieve ϵ_a -link LDP. Specifically, for each bit $a_{v,u} \in \mathbf{a}_v$, the RR mechanism outputs a perturbed value $a'_{v,u}$ according to

$$a'_{v,u} = \begin{cases} a_{v,u}, & \text{with probability } e^{\epsilon_a}/(e^{\epsilon_a} + 1), \\ 1 - a_{v,u}, & \text{with probability } 1/(e^{\epsilon_a} + 1). \end{cases} \quad (4)$$

2.3.2 Server-Side Learning. After collecting perturbed graph data $\{\mathbf{x}'_v, \mathbf{a}'_v\}_{v \in \mathcal{V}}$, the server applies GNNs [69] to learn node representations for downstream tasks. A GNN consists of multiple stacked graph convolution layers, where each layer updates node embeddings by aggregating information from their (perturbed) neighbors and applying a trainable non-linear transformation. Formally, let $\hat{\mathcal{N}}(v)$ denote the neighbor set of node v derived from the perturbed adjacency list $\mathbf{a}'_v$. The embedding of node v at layer ℓ is given by:

$$\mathbf{h}_{\hat{\mathcal{N}}(v)}^{(\ell)} = \text{AGGREGATE}_\ell(\{\mathbf{h}_u^{(\ell-1)} \mid u \in \hat{\mathcal{N}}(v)\}), \quad (5)$$

$$\mathbf{h}_v^{(\ell)} = \text{UPDATE}_\ell(\mathbf{h}_{\hat{\mathcal{N}}(v)}^{(\ell)}), \quad (6)$$

where $\mathbf{h}_u^{(\ell-1)}$ is the embedding of neighbor u at the $(\ell - 1)$ -th layer, $\text{AGGREGATE}_\ell(\cdot)$ is a permutation-invariant function such as mean, sum, or max pooling, and $\text{UPDATE}_\ell(\cdot)$ is a trainable non-linear mapping, typically implemented as a neural network. At the input layer, we initialize $\mathbf{h}_v^{(0)} = \mathbf{x}'_v$ with the perturbed feature vector.

3 Revisiting Attacks

In this section, we revisit prior data poisoning attacks on locally differentially private graph learning under a more realistic *full-privacy* setting. Existing studies [16] primarily consider a *partial-privacy* scenario, where only node features are treated as private and locally perturbed, while adjacency lists are assumed fully observable. Under such assumptions, injected fake nodes and manipulated features or connections can significantly degrade downstream graph learning tasks. However, our empirical and theoretical analysis demonstrates

¹For further details on other LDP mechanisms, please refer to Appendix A.

Table 1: Attack amplification under fully-private settings across four real-world social networks.

Metric	OS LASTFM	TWITCH	GITHUB	FACEBOOK
RRA	1.31	1.24	1.38	1.15

that when both features and adjacency lists are protected by LDP, the impact of these attacks is systematically amplified. In other words, prior evaluations underestimate the potential harm. Sec. 3.1 formalizes the threat model, Sec. 3.2 presents empirical study, and Sec. 3.3 provides theoretical analysis of this amplification effect.

3.1 Threat Model

3.1.1 Adversary Capabilities. We consider an attacker capable of injecting a small set of fake nodes into the original graph $\mathcal{G} = (\mathcal{V}, \mathbf{X}, \mathbf{A})$. The adversary inserts malicious nodes $\mathcal{V}_{\text{atk}}$ with $1 < |\mathcal{V}_{\text{atk}}| \ll |\mathcal{V}|$ and adds adversarial edges $\mathcal{E}_{\text{atk}}$, yielding a poisoned adjacency $\mathbf{A}'$. Let $\mathbf{a}_v$ and $\mathbf{a}'_v$ denote the adjacency rows of node v in $\mathbf{A}$ and $\mathbf{A}'$, respectively. To preserve stealth, the attack is constrained so that each target node $v_t \in \mathcal{V}_t \subset \mathcal{V}$ is incident to at most one adversarial edge. Each injected node $v_{\text{atk}} \in \mathcal{V}_{\text{atk}}$ may specify its raw feature vector and adjacency row prior to local perturbation.

3.1.2 Adversary Knowledge. We assume a practical *black-box setting*: the attacker has access only to the protocol $\mathbb{P}$'s publicly available LDP parameters, which are public and shared among all users. The attacker has no access to genuine users' raw features, labels, exact neighborhoods, or the server-side model weights; in particular, they cannot inspect or query internal states of the server-side GNN beyond outputs or information that the protocol explicitly publishes. A discussion of the attack's realism is provided in Appendix B.

3.1.3 Adversary Objective. Let $\mathcal{G}' = (\mathcal{V} \cup \mathcal{V}_{\text{atk}}, \mathbf{X}', \mathbf{A}')$ denote the poisoned graph after injection (where $\mathbf{X}'$ contains feature rows for injected nodes). Under the protocol $\mathbb{P}$, genuine users apply local perturbation mechanisms to their data before upload; the server then trains a GNN f_θ on the perturbed inputs. The adversary aims to maximize an impact metric $\mathcal{U}(f_\theta(\mathbb{P}(\mathcal{G}')))$ (e.g., increase in test loss or drop in accuracy on target or global sets) while remaining within a stealth budget. This can be written generically as

$$\max_{\mathcal{V}_{\text{atk}}, \mathbf{X}', \mathbf{A}'} \mathcal{U}(f_\theta(\mathbb{P}(\mathcal{G}'))) \quad \text{s.t.} \quad \text{dist}((\mathbf{X}, \mathbf{A}), (\mathbf{X}', \mathbf{A}')) \leq \tau, \quad (7)$$

$$|\mathcal{V}_{\text{atk}}| \leq n_{\text{max}}, \quad \#\{\text{adv. edges incident to } v_t\} \leq 1 \quad \forall v_t \in \mathcal{V}_t,$$

where $\text{dist}(\cdot, \cdot)$ is a problem-dependent distance measuring perturbation magnitude (e.g., a mixed norm on node feature differences and adjacency differences, or an edge-count constraint), τ bounds overall stealthiness, and n_{max} limits the number of injected nodes.

3.2 Empirical Study

To quantify the impact of data poisoning attacks [16] under fully-private settings, we conduct experiments on four real-world social networks: LastFM [50], Twitch [50], Github [49], and Facebook [49]. Node features are perturbed using the PM mechanism [62] with $\epsilon_x = 1.0$, while adjacency lists are protected via RR mechanism [25] under $\epsilon_a = 5.0$. We evaluate attacks using node classification accuracy (ACC%) and define the *Relative Risk of Attack (RRA)* as

$$\text{RRA} = \frac{\text{AE}_{\text{full}}}{\text{AE}_{\text{partial}}}, \quad \text{where} \quad \text{AE} = \frac{\text{ACC}_{\text{clean}} - \text{ACC}_{\text{attacked}}}{\text{ACC}_{\text{clean}}}, \quad (8)$$

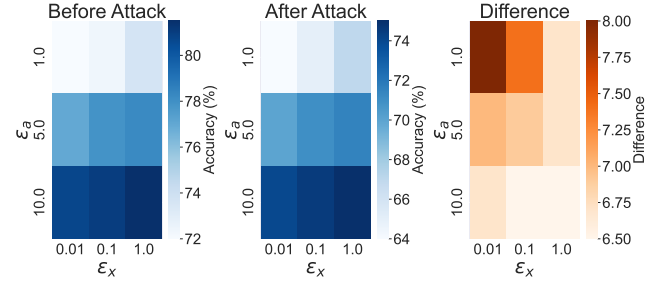


Figure 2: Amplified attack impact under stronger local privacy (The dataset is LastFM): Lower privacy budgets ϵ_x and ϵ_a exacerbate the utility degradation caused by injected nodes.

so that $\text{RRA} > 1$ indicates that attacks are more potent in the fully-private scenario relative to the partial-privacy setting.² The experimental results are summarized in Table 1. Prior attacks that are effective under the partial-privacy assumption exhibit *substantially amplified impact* when both node features and adjacency lists are protected, revealing that earlier evaluations underestimate adversarial reach. We formalize this as follows:

Finding 1. When deployed in a full-privacy setting, data poisoning attacks designed for partial-privacy assumptions achieve larger normalized accuracy drops, demonstrating that full privacy-preserving protocols can inadvertently amplify adversarial influence.

Fig. 2 reports controlled experiments in which we vary the privacy budgets (ϵ_x and ϵ_a) for both features and adjacency (smaller values indicate stronger privacy). The results show that reducing privacy budgets leads to larger attack-induced accuracy drops: stronger LDP increases report noise, inflates observed connectivity of injected nodes, and masks structural cues, thereby facilitating wider propagation of malicious signals under GNN aggregation.

Finding 2. Stricter local privacy (smaller ϵ_x and ϵ_a) amplifies attack impact: under tighter budgets fake nodes obtain inflated observed degrees and induce larger downstream utility degradation.

These empirical observations corroborate our theoretical analysis in Sec. 3.3 and emphasize that full-privacy graph learning, while protecting user data, can unintentionally exacerbate attack severity.

3.3 Theoretical Analysis

We now analyze why the impact of poisoning is amplified in a full-privacy setting. Recalling the perturbation mechanism RR [25, 38] for adjacency lists in Sec. 2.3, each edge indicator a_{vu} is perturbed according to Eq. (4). Fix a node v with true degree $d_v = \sum_{u \neq v} a_{vu}$. The expected reported degree is

$$\begin{aligned} \mathbb{E}[d'_v] &= \sum_{u \neq v} \Pr[a'_{vu} = 1] = p d_v + (1 - p)(|\mathcal{V}| - 1 - d_v) \\ &= d_v + (1 - p)(|\mathcal{V}| - 1 - 2d_v), \end{aligned} \quad (9)$$

where $p = e^{\epsilon_a} / (1 + e^{\epsilon_a})$. The difference

$$\Delta_v = \mathbb{E}[d'_v] - d_v = (1 - p)(|\mathcal{V}| - 1 - 2d_v) \quad (10)$$

is strictly positive whenever $d_v < \frac{|\mathcal{V}| - 1}{2}$, which is the case for most social graphs. Thus RR systematically inflates observed degrees.

²Further experimental details are provided in Sec. 5.1.

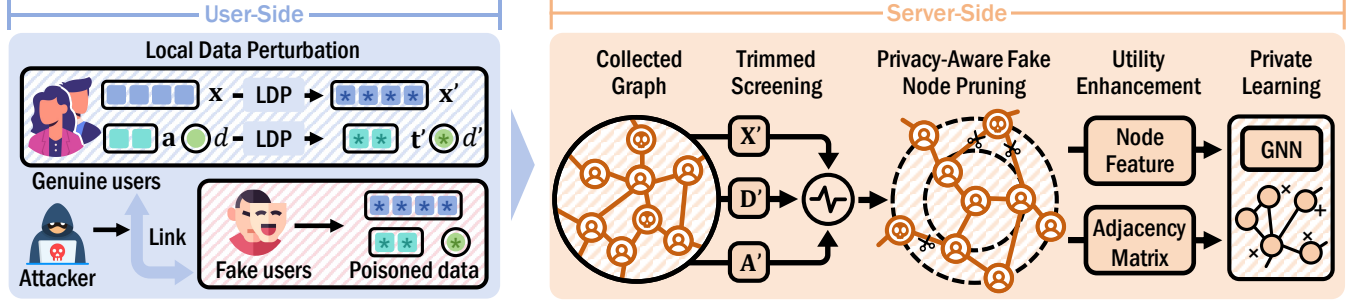


Figure 3: Overview of our proposed CURENET framework, which integrates four coordinated core modules: ❶ local data perturbation, ❷ trimmed screening, ❸ privacy-aware fake node pruning, and ❹ utility enhancement, to effectively defend against data poisoning attacks and foster a robust, secure, and efficient locally private graph learning protocol.

This inflation has two direct consequences for adversaries. *First*, injected fake nodes designed with small degree d_{atk} acquire many additional apparent connections due to RR false positives, leading to an enlarged neighborhood and stronger signal propagation in GNN aggregation. *Second*, because RR injects spurious edges across the graph, adversarial links become less distinguishable from background noise, diminishing the power of structure-based detection. Moreover, the inflation is not only expected but also concentrated. Since $d'_v = \sum_{u \neq v} a'_{vu}$ is a sum of independent Bernoulli variables with mean given in Eq. (24), Hoeffding bounds [21] imply that

$$\Pr [|d_v - \mathbb{E}[d'_v]| \geq t] \leq 2 \exp \left(- \frac{2t^2}{(|V| - 1)} \right). \quad (11)$$

Hence the degree increase is statistically reliable for moderate graph sizes. In summary, as shown in Theorem 3, while RR mechanism protects individual adjacency privacy, it unintentionally inflates degrees and introduces structural noise, both of which expand the reach and concealment of adversarial nodes. These mechanisms explain the empirical amplification observed in our experiments.

Theorem 3. Under randomized response (RR) with privacy parameter $\epsilon_a > 0$, any node v with degree $d_v < (|V| - 1)/2$ satisfies $\mathbb{E}[d'_v] > d_v$. Consequently, a fake node obtains an inflated expected degree, which amplifies its influence in k -hop message passing.

PROOF. Please refer to Appendix C.1 for the detailed proof. $\square$

4 Defense: CURENET

Facing data poisoning threats in locally private graph learning, it is crucial to develop a defense framework that can accurately identify malicious nodes while improving learning utility. Prior studies [15] have shown that existing defenses, such as graph homophily analysis [6, 54, 74, 80] and anomalous node detection [13, 37, 40], are largely ineffective. This limitation arises from the fact that local differential privacy fundamentally obfuscates structural and feature-level correlations, which conventional detection methods rely on, thereby blurring the boundary between benign noise and adversarial perturbation. This observation suggests that robust locally private graph learning requires a fundamental redesign of the protocol, involving careful modification, optimization, and reinforcement of its core components. In this context, we introduce CURENET, a defense framework composed of four core modules: *local data perturbation*, *trimmed screening*, *privacy-aware fake node pruning*, and *utility enhancement*. The overall architecture is illustrated in Fig. 3.

4.1 Local Data Perturbation

To establish a fully-private foundation for server-side defense, each user v locally perturbs their sensitive data prior to uploading. Unlike the data perturbation described in Sec. 2.3, which considers only node features $\mathbf{x}_v \in \mathbb{R}^d$ and adjacency lists $\mathbf{a}_v \in \{0, 1\}^{|V|}$, we additionally include each node's degree $d_v = \|\mathbf{a}_v\|_1$ in the perturbation and collection process. This design enables the server to perform *degree-aware adjustments* on the perturbed adjacency information, which helps calibrate structural signals, improve utility, and mitigate the amplification of data-poisoning attacks under the fully-private setting. Formally, we define the local perturbation as

$$\mathbf{x}'_v = \mathcal{M}_x(\mathbf{x}_v, \epsilon_x), \quad \mathbf{a}'_v = \mathcal{M}_a(\mathbf{a}_v, \epsilon_{a_1}), \quad d'_v = d_v + \text{LP}(1/\epsilon_{a_2}), \quad (12)$$

where $\mathcal{M}_x$ and $\mathcal{M}_a$ denote the LDP mechanisms (e.g., PM [51] for features and RR [25] for adjacency), and Laplace (LP) noise [46] is injected into the degree. The overall privacy budget ϵ is allocated such that ϵ_x is reserved for features, while the structural budget ϵ_a is split into $\epsilon_{a_1} = \rho \epsilon_a$ for the adjacency list and $\epsilon_{a_2} = (1 - \rho) \epsilon_a$ for the degree, with $\rho \in [0, 1]$. This budget partitioning allows the server to leverage noisy degree information for calibrating the perturbed adjacency structure [20, 77], thereby enhancing utility and reducing the amplification of poisoning effects in the fully-private regime. Finally, the server collects the perturbed triplet $(\mathbf{X}', \mathbf{A}', \mathbf{D}') = (\{\mathbf{x}'_v\}_{v \in V}, \{\mathbf{a}'_v\}_{v \in V}, \{d'_v\}_{v \in V})$ and proceeds with poisoning defense and privacy-preserving graph learning. By the following theorem, the overall process is guaranteed to satisfy ϵ -LDP.

Theorem 4. The combined perturbation $(\mathbf{x}'_v, \mathbf{a}'_v, d'_v)$ satisfies ϵ -LDP at the user level with $\epsilon = \epsilon_x + \epsilon_a$, according to the sequential composition (Theorem 1) and post-processing invariance (Theorem 2).

PROOF. Please refer to Appendix C.2 for the detailed proof. $\square$

4.2 Trimmed Screening

The attacker perturbs features by injecting m extreme coordinates $\pm B$ (please refer to [16] for details.) while leaving the remaining $d - m$ coordinates near zero. In contrast, genuine nodes are only subject to local noise with bounded variance. To exploit this discrepancy, we measure each node's distance from a robustly estimated feature center. For each coordinate j , we compute the α -trimmed mean $\hat{\mu}_j$, and define the node-level deviation statistic

$$s_v = (1/\sqrt{d}) \|\mathbf{x}'_v - \hat{\boldsymbol{\mu}}\|_2, \quad \hat{\boldsymbol{\mu}} = (\hat{\mu}_1, \dots, \hat{\mu}_d)^\top. \quad (13)$$

Expanding this term yields $s_v^2 = \frac{1}{d} \sum_{j=1}^d (x'_{v,j} - \hat{\mu}_j)^2$, which represents the normalized average squared deviation across all coordinates. For honest nodes, each $(x'_{v,j} - \hat{\mu}_j)$ is sub-Gaussian with parameter $\bar{\sigma}$, so concentration results imply

$$\Pr[s_v > \bar{\sigma}\sqrt{2\log d/d}] \leq d^{-1}. \quad (14)$$

In contrast, for an adversarial node with m manipulated coordinates, $s_v^2 \approx m/d \cdot B^2$, which is substantially larger whenever $B \gg \bar{\sigma}$. This gap enables a threshold τ^* to be set such that honest nodes fall below τ^* with high probability, while adversarial nodes exceed it. We therefore remove all potentially malicious nodes with $s_v > \tau^*$, obtaining a cleaner graph $\mathcal{G}^*$ for subsequent modules.

Theorem 5. With high probability, honest nodes satisfy $s_v = O(\bar{\sigma})$ while malicious nodes yield $s_v \gtrsim B\sqrt{m/d}$.

PROOF. Please refer to Appendix C.3 for the detailed proof. $\square$

4.3 Privacy-Aware Fake Node Pruning

After the initial screening stage, we obtain a refined graph $\mathcal{G}^*$. Although extreme outliers are removed, sophisticated adversaries may still evade detection by injecting fake nodes with perturbed yet carefully crafted features and links. To counter this, we propose a *privacy-aware pruning mechanism* that leverages the inherent inconsistency between features and structure in malicious insertions. For each node v , we define a *feature-structure consistency score*:

$$\phi(v) = \frac{1}{|\mathcal{N}^*(v)|} \sum_{u \in \mathcal{N}^*(v)} \exp\left(-\frac{\|x'_v - x'_u\|_2^2}{\sigma^2}\right), \quad (15)$$

where x'_v is the LDP-perturbed feature of node v , $\mathcal{N}^*(v)$ denotes its neighborhood in $\mathcal{G}^*$, and σ reflects the perturbation scale of the LDP mechanism. Nodes with low $\phi(v)$ values indicate weak alignment between features and local structure, suggesting potential malicious insertion. We formally establish that fake nodes inevitably exhibit such inconsistency under LDP perturbations, as shown below:

Theorem 6 (Feature-Structure Inconsistency). Let the perturbation mechanism be ϵ -LDP with sub-Gaussian noise of variance σ^2 . For a genuine node v , the expected consistency score satisfies

$$\mathbb{E}[\phi(v)] \geq 1 - O(\sigma^2/\epsilon). \quad (16)$$

For a maliciously injected fake node v_{atk} , whose node features and adjacency lists are independent, the probability of exceeding a pruning threshold δ decays exponentially with its degree:

$$\Pr[\phi(v_{\text{atk}}) \geq \delta] \leq \exp(-c \cdot d_{v_{\text{atk}}}), \quad (17)$$

where $d_{v_{\text{atk}}} = |\mathcal{N}^*(v_{\text{atk}})|$ and $c > 0$ is a constant depending on ϵ .

PROOF. Please refer to Appendix C.4 for the detailed proof. $\square$

Guided by Theorem 6, we prune nodes with $\phi(v) < \delta$, where δ is adaptively chosen based on the privacy budget ϵ to balance robustness and utility. The resulting graph is denoted as $\mathcal{G}^\circ = (\mathcal{V}^\circ, \mathbf{X}^\circ, \mathbf{A}^\circ)$, which serves as the input for the final utility enhancement stage.

4.4 Utility Enhancement

After pruning fake nodes, the remaining graph $\mathcal{G}^\circ$ still suffers from utility degradation due to LDP noise. To address this, we design a *utility enhancement* module that integrates existing feature- and structure-level enhancement methods into a unified framework.

Rather than proposing a new mechanism, our goal is to provide a protocol-level abstraction that accommodates diverse utility optimization techniques. Formally, let $\mathbf{X}^\circ$ denote the features and $\mathbf{A}^\circ$ the sanitized adjacency matrix of $\mathcal{G}^\circ$. We define a generic enhancement operator $\mathcal{U}(\cdot)$ that jointly refines features and structure:

$$(\mathbf{X}^\circ, \mathbf{A}^\circ) = \mathcal{U}(\mathbf{X}^\circ, \mathbf{A}^\circ; \Psi_x, \Psi_a), \quad (18)$$

where $\mathbf{X}^\circ$ and $\mathbf{A}^\circ$ are the enhanced inputs for downstream graph learning. Ψ_x and Ψ_a correspond to feature enhancement (e.g., aggregation denoising [15, 38, 51]) and structure enhancement (e.g., degree protection [20, 77]) modules, respectively. We formulate the joint utility enhancement as the following optimization problem:

$$\min_{\Psi_x, \Psi_a} \mathcal{L}_{\text{task}}(f_\theta(\mathbf{X}^\circ, \mathbf{A}^\circ), \mathbf{Y}) + \lambda \mathcal{R}(\mathbf{X}^\circ, \mathbf{A}^\circ), \quad (19)$$

where $\mathcal{L}_{\text{task}}$ is the downstream learning loss, and $\mathcal{R}$ is a regularization term encouraging consistency with the LDP constraints. This abstraction allows CURENET to serve as a unifying substrate for feature- and structure-level utility remedies, while remaining adaptable to future advances in LDP-aware optimization.

5 Experiments

In this section, we conduct a series of experiments to assess the effectiveness of our method. Sec. 5.1 describes the experimental setup, and Sec. 5.2 presents the results along with detailed analysis.

5.1 Experimental Settings

5.1.1 Datasets. We conduct experiments on four social networks, including LastFM [50], Twitch [50], Github [49], and Facebook [49], which have been broadly adopted for evaluating graph learning methods. LastFM is used by default. Table 3 summarizes their statistics, and detailed descriptions are provided in Appendix D.1.

5.1.2 GNN Models. We consider six representative GNN models: GCN [28], GraphSAGE [14], GAT [61], APPNP [30], SGC [66], and SSGC [76]. Unless otherwise specified, each model is implemented with two graph convolutional layers, where the hidden dimension is set to 64. ReLU activation [29] and dropout [2] are applied throughout. By default, GCN serves as the backbone model.

5.1.3 LDP Mechanisms. To protect node features, we evaluate six representative LDP mechanisms. Among them, three are recent state-of-the-art methods: the piecewise mechanism (PM) [45, 62], the multi-bit mechanism (MB) [38, 51], and the square wave mechanism (SW) [34, 35]. In addition, we include three classical baselines: the 1-bit mechanism (1B) [7], the Laplace mechanism (LP) [46], and the Analytic Gaussian mechanism (AG) [3]. For adjacency protection, we adopt the randomized response (RR) [20, 25, 38, 47, 77]. Unless otherwise specified, PM is used as the default feature perturbation mechanism. Further details are provided in Appendix A.

5.1.4 Parameter Settings. All datasets are randomly split into 50% training, 25% validation, and 25% test sets. For node feature privacy, the LDP budget ϵ_x is varied over $\{0.01, 0.1, 1.0\}$, while for adjacency list privacy, the budget ϵ_a is set to $\{1.0, 5.0, 10.0\}$. Following prior works [4, 16, 19, 22, 33, 36, 58, 60, 68], the proportion of maliciously injected nodes r^* is varied over $\{0.001, 0.005, 0.01, 0.02, 0.04\}$. By default, $\epsilon_x = 0.1$, $\epsilon_a = 5$, and $r^* = 0.01$. The thresholds τ^* and δ are set to $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, respectively. ϱ is set to 0.5. In the utility

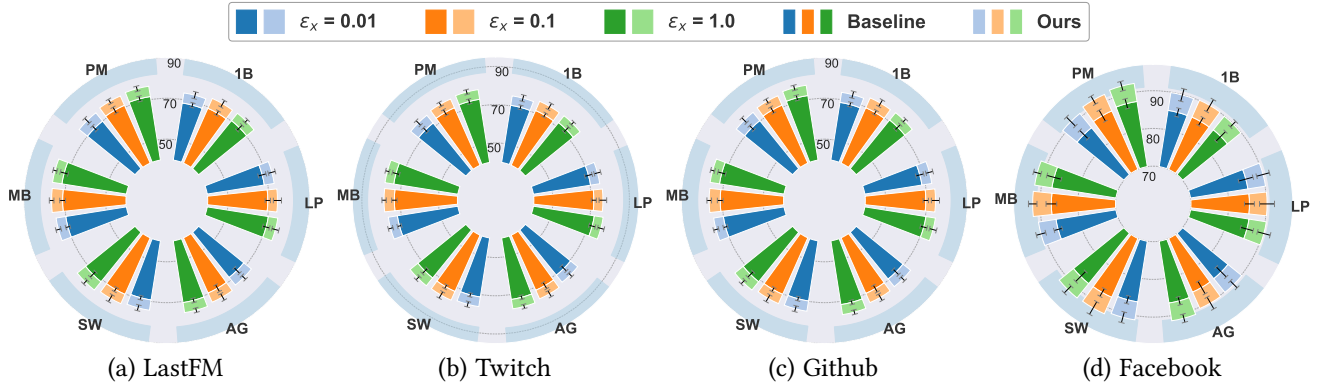


Figure 4: Performance comparison between our proposed CURENET (Ours) and attack baseline on the node classification task.

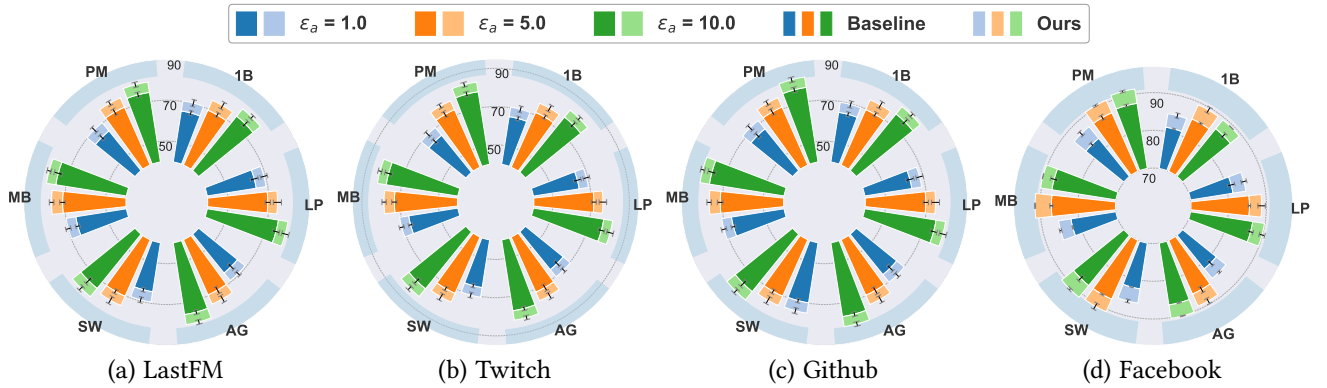


Figure 5: Performance comparison between our proposed CURENET (Ours) and attack baseline on the node classification task.

enhancement module, Ψ_x and Ψ_a adopt feature- and structure-level enhancement techniques from [15] and [77], respectively.

5.1.5 Evaluation Metrics. We assess performance on two core graph learning tasks: *node classification* and *link prediction*, which are widely used to benchmark graph representation methods and support downstream applications. For node classification, we measure test set *accuracy*, while for link prediction we report *AUC* (area under the ROC curve). Results are averaged over 10 independent runs, with 95% confidence intervals computed via 1000-fold bootstrapped resampling. Node classification is considered the default task.

5.2 Experimental Results

5.2.1 Effectiveness. We thoroughly evaluate the effectiveness of our defense across four benchmark datasets under varying noise scales (ϵ_x and ϵ_a), and compare the results with the attack baseline. Figures 4 and 5 illustrate the node classification performance under different privacy budgets, where CURENET consistently achieves superior accuracy across all settings. This consistent improvement demonstrates that our defense effectively mitigates the impact of malicious nodes. Moreover, the performance gap between CURENET and the attack baseline widens as the noise level increases, highlighting its resilience to stronger privacy constraints and heavier perturbations. Table 2 further confirms the generalizability of our framework across diverse GNN architectures, including GCN, GraphSAGE, GAT, APPNP, SGC, and SSGC. Additionally, link prediction results provided in Appendix D.2 reinforce our findings

Table 2: Comparison of classification accuracy between our defense and attack baseline across different GNN models.

METHOD	GCN	GRAGHSAGE	GAT	APPNP	SGC	SSGC
BASELINE	71.5	72.1	69.8	72.2	72.0	71.9
OURS	75.7	75.9	74.8	76.0	75.8	75.6

by showing that CURENET also improves robustness in edge-level inference tasks. Overall, these results demonstrate that our defense substantially enhances the utility and stability of locally private graph learning, effectively countering data poisoning threats across various models, datasets, and privacy levels.

5.2.2 Ablation Studies. To examine the contribution of each defense component in CURENET, we conduct an ablation study on the LastFM dataset under different configurations, including **Non-Priv**, a non-private baseline without noise injection; **NoAttack**, trained on clean data without adversarial poisoning; **Attacked**, representing the case without any defense; the complete **CURENET** framework; and three ablated variants, where each core module is removed in turn: **w/o S** (removing the *Trimmed Screening* module), **w/o P** (removing the *Privacy-Aware Fake Node Pruning* module), and **w/o E** (removing the *Utility Enhancement* module). As illustrated in Fig. 6(a), removing any module results in a noticeable drop in performance, while the full CURENET achieves the best balance among privacy, utility, and robustness, demonstrating the complementary roles of the three core defense components.

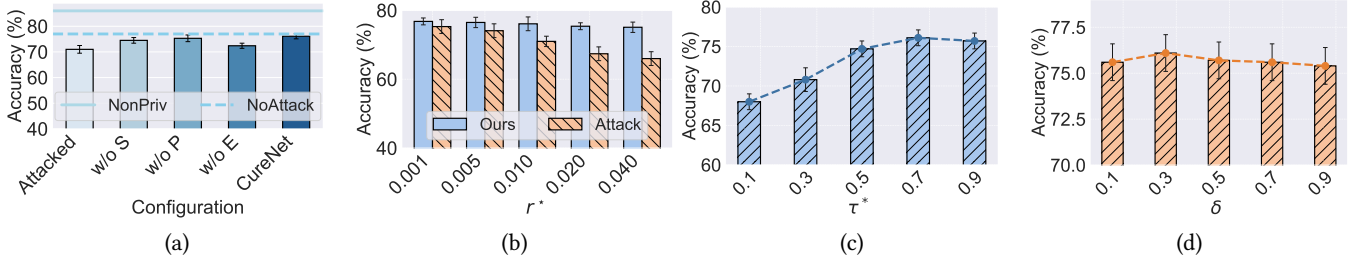


Figure 6: (a) Ablation study showing the contribution of each module. (b–d) Sensitivity of defense performance to key parameters: (b) the proportion of malicious nodes r^* , (c) the trimmed screening threshold τ^* , and (d) the privacy-aware pruning threshold δ .

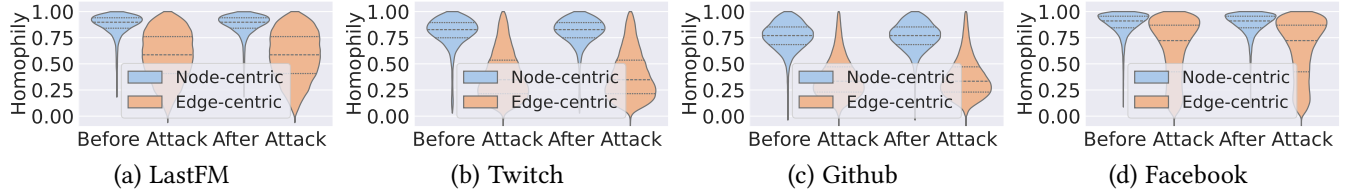


Figure 7: Comparison of graph homophily distributions across four datasets before and after the attack.

5.2.3 Other Defenses. Graph attacks can disrupt the homophily structure of a network, and homophily-based defenses have been proposed to detect such manipulations [6, 54, 74, 80]. We evaluate these defenses on all datasets before and after our attack using two metrics [6]: *node-centered homophily*, measuring similarity between a node and its neighbors, and *edge-centered homophily*, measuring similarity across connected node pairs. As shown in Fig. 7, our attack preserves overall homophily, producing only subtle changes that do not trigger these defenses. Node-centered homophily remains higher than edge-centered homophily across datasets. These results demonstrate that standard homophily-based defenses are ineffective against our poisoning attack method, emphasizing the need for specialized protections in fully-private graph learning.

5.2.4 Parameter Analysis. We conduct a parameter sensitivity study on three key variables, as shown in Fig. 6(b–d). The proportion of maliciously injected nodes r^* is varied over $\{0.001, 0.005, 0.01, 0.02, 0.04\}$, and the results confirm the robustness of our defense against increasing attack intensity. The trimmed screening threshold τ^* and the privacy-aware pruning threshold δ are independently varied over $\{0.1, 0.3, 0.5, 0.7, 0.9\}$ to examine their influence on defense effectiveness and utility preservation. The results indicate that selecting appropriate values for τ^* and δ is crucial for achieving optimal defense performance, as they balance the trade-off between removing malicious nodes and retaining genuine graph structure.

6 Related Works

In this section, we briefly review two areas closely related to our study: *data poisoning attacks on GNNs* and *locally private graph learning*. For a more detailed discussion, please see Appendix E.

6.0.1 Data Poisoning Attacks on GNNs. Data poisoning attacks aim to disrupt the training of machine learning models by injecting carefully crafted malicious data. In GNNs, such attacks are particularly effective because of the complex dependencies among nodes, edges, and overall graph topology that these models leverage for representation learning [54, 78–80]. Existing studies primarily consider

conventional GNN settings with full access to node features and adjacency information. By contrast, attacks in locally private graph learning face stricter constraints: nodes and edges are perturbed locally, the attacker has limited knowledge of the true graph, and the protocol adds structural obfuscation. These conditions require novel strategies to meaningfully degrade model utility.

6.0.2 Locally Private Graph Learning. Locally private graph learning protocols generalize LDP to graph-structured data, enabling each user to perturb their sensitive features and adjacency information before uploading [15, 16, 20, 34, 38, 45, 47, 51, 77]. By leveraging multi-layer message passing in GNNs [28, 69], these protocols can tolerate noisy inputs and still produce useful embeddings for downstream tasks. However, the decentralized and obfuscated nature of locally private graph learning creates new attack surfaces, making them vulnerable to malicious injections that exploit both feature and structural perturbations. This underscores the need for defense strategies specifically designed for locally private graph learning.

7 Conclusion

This work systematically investigates the threat of data poisoning in fully-private graph learning, revealing that privacy mechanisms can unintentionally amplify attack impact. To address this challenge, we propose CURENET, a unified defense framework that integrates four core modules: ① local data perturbation, ② trimmed screening, ③ privacy-aware fake node pruning, and ④ utility enhancement. Extensive experiments on multiple real-world social networks demonstrate that CURENET effectively mitigates poisoning attacks while preserving high utility. Our study highlights the urgent need for robust defenses in privacy-preserving graph learning and provides a practical solution for securing LDP-based protocols.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (No. 2024YFF0907401), the National Natural Science Foundation of China (No. 62072052), and the Shandong Provincial Natural Science Foundation (No. ZR2025MS1038).

References

- [1] Sami Abu-El-Hajja, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *International Conference on Machine Learning (ICML)*. 21–29.
- [2] Pierre Baldi and Peter J Sadowski. 2013. Understanding dropout. In *Advances in Neural Information Processing Systems (NeurIPS)*. 2814–2822.
- [3] Borja Balle and Yu-Xiang Wang. 2018. Improving the gaussian mechanism for differential privacy: Analytical calibration and optimal denoising. In *International Conference on Machine Learning (ICML)*. 394–403.
- [4] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. Data poisoning attacks to local differential privacy protocols. In *30th USENIX Security Symposium (USENIX Security)*. 947–964.
- [5] Christopher J Carpenter. 2012. Narcissism on Facebook: Self-promotional and anti-social behavior. *Personality and Individual Differences* 52, 4 (2012), 482–486.
- [6] Yongqiang Chen, Han Yang, Yonggang Zhang, Ma Kaili, Tongliang Liu, Bo Han, and James Cheng. 2022. Understanding and Improving Graph Injection Attack by Promoting Unnoticeability. In *International Conference on Learning Representations (ICLR)*. 1–13.
- [7] Bolin Ding, Janardhan Kulkarni, and Sergey Yekhanin. 2017. Collecting telemetry data privately. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3571–3580.
- [8] John R Douceur. 2002. The sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*. 251–260.
- [9] Cynthia Dwork. 2008. Differential privacy: A survey of results. In *International Conference on Theory and Applications of Models of Computation (TMAC)*. 1–19.
- [10] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Third Theory of Cryptography Conference (TCC)*. 265–284.
- [11] Úlfar Erlingsson, Vasily Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1054–1067.
- [12] Jiarui Feng, Yixin Chen, Fuhai Li, Anindya Sarkar, and Muhan Zhang. 2022. How powerful are k-hop message passing graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 4776–4790.
- [13] Michelle Girvan and Mark EJ Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826.
- [14] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*. 1024–1034.
- [15] Longzhu He, Chaozhuo Li, Peng Tang, and Sen Su. 2025. Going Deeper into Locally Differentially Private Graph Neural Networks. In *Forty-second International Conference on Machine Learning (ICML)*. 22548–22565.
- [16] Longzhu He, Chaozhuo Li, Peng Tang, Li Sun, Sen Su, and Philip S Yu. 2025. Devil’s Hand: Data Poisoning Attacks to Locally Private Graph Learning Protocols. *arXiv preprint arXiv:2506.09803* (2025).
- [17] Longzhu He, Peng Tang, Yuanhe Zhang, Pengpeng Zhou, and Sen Su. 2025. Mitigating privacy risks in Retrieval-Augmented Generation via locally private entity perturbation. *Information Processing & Management* 62, 4 (2025), 104150.
- [18] Longgang He and Zhenhua Wei. 2026. Towards structure-aware data augmentation for high-degree graph neural networks. *Information Processing & Management* 63, 1 (2026), 104343.
- [19] Xi He, Kai Huang, Qingqing Ye, and Haibo Hu. 2025. Data Poisoning Attacks to Local Differential Privacy Protocols for Graphs. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 987–1000.
- [20] Seira Hidano and Takao Murakami. 2024. Degree-Preserving Randomized Response for Graph Neural Networks under Local Differential Privacy. *Transactions on Data Privacy* 17, 2 (2024), 89–121.
- [21] Wassily Hoeffding. 1963. Probability inequalities for sums of bounded random variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30.
- [22] I Hsu, Chih-Hsun Lin, Chia-Mu Yu, Sy-Yen Kuo, Chun-Ying Huang, et al. 2025. Poisoning Attacks to Local Differential Privacy Protocols for Trajectory Data. *arXiv preprint arXiv:2503.07483* (2025).
- [23] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally differentially private analysis of graph statistics. In *30th USENIX Security Symposium (USENIX Security)*. 983–1000.
- [24] Hongwei Jin and Xun Chen. 2022. Gromov-Wasserstein Discrepancy with Local Differential Privacy for Distributed Structural Graphs. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI)*. 2115–2121.
- [25] Peter Kairouz, Keith Bonawitz, and Daniel Ramage. 2016. Discrete distribution estimation under local privacy. In *International Conference on Machine Learning (ICML)*. 2436–2444.
- [26] Peter Kairouz, Sewoong Oh, and Pramod Viswanath. 2017. The Composition Theorem for Differential Privacy. *IEEE Transactions on Information Theory* 63, 6 (2017), 4037–4049.
- [27] Zong Ke, Yuqing Cao, Zhenrui Chen, Yuchen Yin, Shouchao He, and Yu Cheng. 2025. Early warning of cryptocurrency reversal risks via multi-source data. *Finance Research Letters* (2025), 107890.
- [28] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*. 1–14.
- [29] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. 2017. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 971–980.
- [30] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *7th International Conference on Learning Representations (ICLR)*. 1–15.
- [31] Issie Lapowsky. 2019. NATO group catfished soldiers to prove a point about privacy. *Wired* (2019).
- [32] Pu Li, Xiaoyan Yu, Hao Peng, Yantuan Xian, Linqin Wang, Li Sun, Jingyun Zhang, and Philip S Yu. 2024. Relational prompt-based pre-trained language models for social event detection. *ACM Transactions on Information Systems* 43, 1 (2024), 1–43.
- [33] Xiaoguang Li, Ninghui Li, Wenhui Sun, Neil Zhenqiang Gong, and Hui Li. 2023. Fine-grained poisoning attack to local differential privacy protocols for mean and variance estimation. In *32nd USENIX Security Symposium (USENIX Security)*. 1739–1756.
- [34] Zening Li, Rong-Hua Li, Meihao Liao, Fusheng Jin, and Guoren Wang. 2024. Privacy-Preserving Graph Embedding based on Local Differential Privacy. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. 1316–1325.
- [35] Zitao Li, Tianhao Wang, Milan Lopuhaä-Zwakenberg, Ninghui Li, and Boris Škoric. 2020. Estimating numerical distributions under local differential privacy. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 621–635.
- [36] Ting-Wei Liao, Chih-Hsun Lin, Yu-Lin Tsai, Takao Murakami, Chia-Mu Yu, Jun Sakuma, Chun-Ying Huang, and Hiroaki Kikuchi. 2025. Data Poisoning Attacks to Locally Differentially Private Range Query Protocols. *arXiv preprint arXiv:2503.03454* (2025).
- [37] Aristidis Likas, Nikos Vlassis, and Jakob J Verbeek. 2003. The global k-means clustering algorithm. *Pattern Recognition* 36, 2 (2003), 451–461.
- [38] Wanyu Lin, Baohun Li, and Cong Wang. 2022. Towards private learning on decentralized graphs with local differential privacy. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2936–2946.
- [39] Jiaqi Ma, Shuangrui Ding, and Qiaozhu Mei. 2020. Towards more practical adversarial attacks on graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 4756–4766.
- [40] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2021), 12012–12038.
- [41] Lingshuo Meng, Yijie Bai, Yanjiao Chen, Yutong Hu, Wenyuan Xu, and Haiqin Weng. 2023. Devil in Disguise: Breaching Graph Neural Networks Privacy through Infiltration. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1153–1167.
- [42] Michele Nitti, Luigi Atzori, and Irena Pletikosa Cvijikj. 2014. Friendship selection in the social internet of things: challenges and possible strategies. *IEEE Internet of Things Journal* 2, 3 (2014), 240–247.
- [43] Kaichen Ouyang, Zong Ke, Shengwei Fu, Lingjie Liu, Puning Zhao, and Dayu Hu. 2024. Learn from global correlations: Enhancing evolutionary algorithm via spectral gnn. *arXiv preprint arXiv:2412.17629* (2024).
- [44] Kaichen Ouyang, Mingyang Yu, Zong Ke, Junbo Jacob Lian, Shengwei Fu, Xiaoyang Hao, Shengju Yu, and Dayu Hu. 2025. Wasserstein Evolution: Evolutionary Optimization as Phase Transition. *arXiv preprint arXiv:2512.05837* (2025).
- [45] Xinjun Pei, Xiaoheng Deng, Shengwei Tian, Jianqing Liu, and Kaiping Xue. 2023. Privacy-enhanced graph neural network for decentralized local graphs. *IEEE Transactions on Information Forensics and Security* 19 (2023), 1614–1629.
- [46] NhatHai Phan, Xintao Wu, Han Hu, and Dejing Dou. 2017. Adaptive laplace mechanism: Differential privacy preservation in deep learning. In *2017 IEEE International Conference on Data Mining (ICDM)*. 385–394.
- [47] Yuxin Qi, Xi Lin, Ziyao Liu, Gaolei Li, Jingyu Wang, and Jianhua Li. 2024. Link-Guard: Link Locally Privacy-Preserving Graph Neural Networks with Integrated Denoising and Private Learning. In *Companion Proceedings of the ACM on Web Conference 2024 (WWW)*. 593–596.
- [48] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 425–438.
- [49] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-scale attributed node embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.
- [50] Benedek Rozemberczki and Rik Sarkar. 2020. Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM)*. 1325–1334.

- [51] Sina Sajadmanesh and Daniel Gatica-Perez. 2021. Locally private graph neural networks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2130–2145.
- [52] Thaler Shannon. 2023. Scientists found over 1,000 AI bots on X stealing selfies to create fake accounts. *New York Post* (2023).
- [53] Yun Shen, Xinlei He, Yufei Han, and Yang Zhang. 2022. Model stealing attacks against inductive graph neural networks. In *2022 IEEE Symposium on Security and Privacy (SP)*. 1175–1192.
- [54] Lichao Sun, Yingdong Dou, Carl Yang, Kai Zhang, Ji Wang, S Yu Philip, Lifang He, and Bo Li. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 7693–7711.
- [55] Li Sun, Zhenhao Huang, Qiqi Wan, Hao Peng, and Philip S Yu. 2024. Spiking graph neural network on riemannian manifolds. In *Advances in Neural Information Processing Systems (NeurIPS)*. 34025–34055.
- [56] Li Sun, Zhenhao Huang, Suyang Zhou, Qiqi Wan, Hao Peng, and Philip Yu. 2025. Riemannfmm: Learning a graph foundation model from riemannian geometry. In *Proceedings of the ACM on Web Conference 2025 (WWW)*. 1154–1165.
- [57] Li Sun, Zhongbao Zhang, Pengxin Ji, Jian Wen, Sen Su, and Philip S Yu. 2019. DNA: Dynamic social network alignment. In *2019 IEEE International Conference on Big Data (BigData)*. 1224–1231.
- [58] Xinyue Sun, Qingqing Ye, Haibo Hu, Jiawei Duan, Tianyu Wo, Jie Xu, and Renyu Yang. 2024. Ldprecover: Recovering frequencies from poisoning attacks against local differential privacy. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 1619–1631.
- [59] Abhradeep Guha Thakurta, Andrew H Vyrros, Umesh S Vaishampayan, Gaurav Kapoor, Julien Freudiger, Vivek Rangarajan Sridhar, and Doug Davidson. 2017. Learning new words, US Patent 9,594,741.
- [60] Wei Tong, Haoyu Chen, Jiacheng Niu, and Sheng Zhong. 2024. Data poisoning attacks to locally differentially private frequent itemset mining protocols. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 3555–3569.
- [61] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*. 1–12.
- [62] Ning Wang, Xiaokui Xiao, Yin Yang, Jun Zhao, Siu Cheung Hui, Hyejin Shin, Junbum Shin, and Ge Yu. 2019. Collecting and analyzing multidimensional data with local differential privacy. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 638–649.
- [63] Xiuling Wang and Wendy Hui Wang. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2871–2884.
- [64] Bang Wu, Xiangwen Yang, Shirui Pan, and Xingliang Yuan. 2022. Model extraction attacks on graph neural networks: Taxonomy and realisation. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security (ASIACCS)*. 337–350.
- [65] Fan Wu, Yunhui Long, Ce Zhang, and Bo Li. 2022. Linkteller: Recovering private edges from graph neural networks via influence analysis. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2005–2024.
- [66] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International conference on machine learning (ICML)*. 6861–6871.
- [67] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: a survey. *Comput. Surveys* 55, 5 (2022), 1–37.
- [68] Yongji Wu, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2022. Poisoning attacks to local differential privacy protocols for Key-Value data. In *31st USENIX Security Symposium (USENIX Security)*. 519–536.
- [69] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2020), 4–24.
- [70] Xingxing Xiong, Shubo Liu, Dan Li, Zhaohui Cai, and Xiaoguang Niu. 2020. A comprehensive survey on local differential privacy. *Security and Communication Networks* 2020, 1 (2020), 8829523.
- [71] Mengmeng Yang, Taolin Guo, Tianqing Zhu, Ivan Tjuawinata, Jun Zhao, and Kwok-Yan Lam. 2024. Local differential privacy and its applications: A comprehensive survey. *Computer Standards & Interfaces* 89 (2024), 103827.
- [72] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (SIGKDD)*. 974–983.
- [73] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 5171–5181.
- [74] Xiang Zhang and Marinka Zitnik. 2020. GnnGuard: Defending graph neural networks against adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*. 9263–9275.
- [75] Zhikun Zhang, Min Chen, Michael Backes, Yun Shen, and Yang Zhang. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security)*. 4543–4560.
- [76] Hao Zhu and Piotr Koniusz. 2021. Simple Spectral Graph Convolution. In *International Conference on Learning Representations (ICLR)*. 1–15.
- [77] Xiaochen Zhu, Vincent YF Tan, and Xiaokui Xiao. 2023. Blink: Link Local Differential Privacy in Graph Neural Networks via Bayesian Estimation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2651–2664.
- [78] Xu Zou, Qinkai Zheng, Yuxiao Dong, Xinyu Guan, Evgeny Kharlamov, Jialiang Lu, and Jie Tang. 2021. Tdgia: Effective injection attacks on graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (SIGKDD)*. 2461–2471.
- [79] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (SIGKDD)*. 2847–2856.
- [80] Daniel Zügner, Oliver Borchert, Amir Akbarnejad, and Stephan Günnemann. 2020. Adversarial attacks on graph neural networks: Perturbations and their patterns. *ACM Transactions on Knowledge Discovery from Data* 14, 5 (2020), 1–31.

A Advanced LDP Mechanisms

In Sec. 2.3, we discuss three advanced LDP mechanisms that are specifically designed for the protection of node features: the piecewise mechanism (PM) [45, 62], the multi-bit mechanism (MB) [24, 38, 51], and the square wave mechanism (SW) [34, 35]. Below, we present the formulations for the MB and SW mechanisms:

- **MB:** This mechanism operates in a one-dimensional space, perturbing a real-valued input $x \in [\alpha, \beta]$ by probabilistically selecting a binary output $x' \in \{-1, 1\}$. The probability distribution function for x' given x is:

$$\Pr[x' = c \mid x] = \begin{cases} \frac{1}{e^\epsilon + 1} + \frac{x - \alpha}{\beta - \alpha} \cdot \frac{e^\epsilon - 1}{e^\epsilon + 1}, & \text{if } c = 1 \\ \frac{e^\epsilon}{e^\epsilon + 1} - \frac{x - \alpha}{\beta - \alpha} \cdot \frac{e^\epsilon - 1}{e^\epsilon + 1}, & \text{if } c = -1 \end{cases}. \quad (20)$$

MB ensures ϵ -LDP while preserving the relative position of the input by using a probabilistic encoding with two discrete values.

- **SW:** This mechanism also operates in one dimension, but it perturbs the input $x \in [\alpha, \beta]$ by expanding the domain to $[-b - 1, b + 1]$, where b is defined as:

$$b = \frac{e^\epsilon - e^\epsilon + 1}{e^\epsilon (e^\epsilon - e^\epsilon - 1)}. \quad (21)$$

The output x' is then sampled according to the following distribution:

$$\Pr[x' = c \mid x] = \begin{cases} p, & \text{if } c \in [x - b, x + b] \\ p/e^\epsilon, & \text{if } c \in [-b - 1, x - b) \cup (x + b, b + 1] \end{cases}, \quad (22)$$

where $p = \frac{e^\epsilon}{2be^\epsilon + 2}$. This mechanism creates a localized uniform distribution centered around x , with exponentially decaying probabilities for values farther away.

B Attack Practicality

The attack model is highly feasible in real-world scenarios, where adversaries can easily create fake accounts and inject them into social networks [8, 31, 52]. By sending friend requests to genuine users, attackers can establish connections without significant resources, as many users are motivated by social influence to accept such requests [5, 42]. This has been demonstrated in prior studies [54, 78, 79], and the dynamics of evolving social graphs make fake node injection a realistic threat during model training.

Table 3: Statistics of graph datasets

DATASET	#NODES	#EDGES	#FEATURES	#CLASSES
CS LASTFM [50]	7,624	27,806	7,842	18
TWITCH [50]	4,648	61,706	128	2
GITHUB [49]	37,700	289,003	4,005	2
f FACEBOOK [49]	22,470	170,912	4,714	4

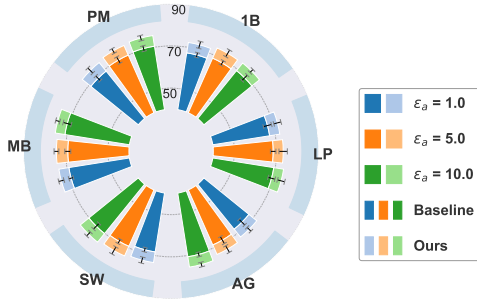


Figure 8: Performance comparison between our proposed CURENET (Ours) and attack baseline on the link prediction.

C Theoretical Proof

C.1 Proof of Theorem 3

PROOF. To prove Theorem 3, we start by recalling the randomized response (RR) [25] mechanism for perturbing the adjacency list, given by the following:

$$a'_{v,u} = \begin{cases} a_{v,u}, & \text{with probability } p = \frac{e^{\epsilon_a}}{e^{\epsilon_a} + 1}, \\ 1 - a_{v,u}, & \text{with probability } 1 - p = \frac{1}{e^{\epsilon_a} + 1}. \end{cases} \quad (23)$$

Let $d_v = \sum_{u \neq v} a_{vu}$ be the true degree of node v , i.e., the number of edges incident to node v . The expected degree after applying the perturbation mechanism is given by:

$$\begin{aligned} \mathbb{E}[d'_v] &= \sum_{u \neq v} \Pr[a'_{vu} = 1] = p \cdot d_v + (1 - p) \cdot (|\mathcal{V}| - 1 - d_v), \\ &= d_v + (1 - p)(|\mathcal{V}| - 1 - 2d_v), \end{aligned} \quad (24)$$

where $p = \frac{e^{\epsilon_a}}{e^{\epsilon_a} + 1}$. The difference between the expected reported degree and the true degree is:

$$\Delta_v = \mathbb{E}[d'_v] - d_v = (1 - p)(|\mathcal{V}| - 1 - 2d_v). \quad (25)$$

Next, we show that for $d_v < \frac{|\mathcal{V}| - 1}{2}$, the difference $\Delta_v > 0$, meaning the expected degree $\mathbb{E}[d'_v]$ is strictly greater than the true degree d_v . To do so, observe that the term $(1 - p)$ is always positive since $0 < p < 1$ for $\epsilon_a > 0$. Thus, the sign of Δ_v is determined by the term $(|\mathcal{V}| - 1 - 2d_v)$. For most social graphs, it is reasonable to assume that $d_v < \frac{|\mathcal{V}| - 1}{2}$, meaning that:

$$|\mathcal{V}| - 1 - 2d_v > 0. \quad (26)$$

Therefore, $(|\mathcal{V}| - 1 - 2d_v)$ is positive, and since $(1 - p)$ is also positive, we have:

$$\Delta_v > 0. \quad (27)$$

This proves that $\mathbb{E}[d'_v] > d_v$ whenever $d_v < \frac{|\mathcal{V}| - 1}{2}$, as required. Consequently, the expected degree of any node with degree less

than half the total number of nodes is inflated, which amplifies the influence of fake nodes in k -hop message passing algorithms. This amplification occurs because the inflated degree leads to a larger number of neighbors in subsequent hops, which increases the node's influence in the graph-based model. $\square$

C.2 Proof of Theorem 4

PROOF. The original node feature $\mathbf{x}$ is perturbed to $\mathbf{x}'$ via $\mathcal{M}_x$ (e.g., using the MB mechanism [51]), and the adjacency list $\mathbf{a}$ is perturbed to $\mathbf{a}'$ via $\mathcal{M}_a$ (e.g., using the RR mechanism [38]). Since the proofs of LDP satisfaction for $\mathcal{M}_x$ and $\mathcal{M}_a$ have been provided in previous works [38, 51, 77], we omit further details. Therefore, considering the perturbations to node features, adjacency lists, and node degrees, the perturbation process satisfies ϵ -LDP by the sequential composition theorem. Additionally, post-processing invariance ensures that operations on the perturbed data (such as graph learning or defense mechanisms) preserve the ϵ -LDP guaranty. $\square$

C.3 Proof of Theorem 5

PROOF. To prove Theorem 5, we analyze the node-level deviation statistic s_v for both genuine and malicious nodes.

For genuine nodes, each feature coordinate $x'_{v,j}$ is subject to local noise with bounded variance, and the deviation from the α -trimmed mean $\hat{\mu}_j$ is sub-Gaussian with parameter $\bar{\sigma}$. Thus, the node-level deviation statistic is:

$$s_v^2 = \frac{1}{d} \sum_{j=1}^d (x'_{v,j} - \hat{\mu}_j)^2. \quad (28)$$

By concentration results for sub-Gaussian variables, we have:

$$\Pr \left[s_v > \bar{\sigma} \sqrt{\frac{2 \log d}{d}} \right] \leq d^{-1}, \quad (29)$$

so $s_v = O(\bar{\sigma})$ for honest nodes.

For malicious nodes, the attacker manipulates m coordinates by injecting extreme values $\pm B$, while the remaining coordinates are near zero. The deviation statistic for malicious nodes is: $s_v^2 \approx \frac{m}{d} \cdot B^2$. Thus, $s_v \gtrsim B\sqrt{\frac{m}{d}}$ for malicious nodes.

The deviation statistic for honest nodes is $s_v = O(\bar{\sigma})$, while for malicious nodes, $s_v \gtrsim B\sqrt{m/d}$. The gap between these values allows us to set a threshold τ^* such that honest nodes have $s_v < \tau^*$ and malicious nodes exceed it. Therefore, we can filter out malicious nodes and obtain a cleaner graph $\mathcal{G}^*$. Thus, we have shown that with high probability, honest nodes satisfy $s_v = O(\bar{\sigma})$ and malicious nodes yield $s_v \gtrsim B\sqrt{m/d}$. $\square$

C.4 Proof of Theorem 6

PROOF. We now prove Theorem 6 by analyzing the feature-structure consistency score $\phi(v)$ for both genuine and malicious nodes under ϵ -LDP perturbations.

For a genuine node v , the perturbation to its feature vector $\mathbf{x}'_v$ is caused by local differential privacy (LDP) noise. Given that the noise is sub-Gaussian with variance σ^2 , the feature vector $\mathbf{x}'_v$ is close to the original feature vector $\mathbf{x}_v$. Thus, the consistency score

$\phi(v)$ reflects the alignment between the feature vector of node v and its neighbors. The expected consistency score is computed as:

$$\mathbb{E}[\phi(v)] = \frac{1}{|\mathcal{N}^*(v)|} \sum_{u \in \mathcal{N}^*(v)} \mathbb{E} \left[\exp \left(-\frac{\|\mathbf{x}'_v - \mathbf{x}'_u\|_2^2}{\sigma^2} \right) \right]. \quad (30)$$

Since the noise is sub-Gaussian with variance σ^2 , the expected value of the exponential term is close to 1, leading to: $\mathbb{E}[\phi(v)] \geq 1 - O\left(\frac{\sigma^2}{\epsilon}\right)$. This shows that genuine nodes exhibit strong feature-structure consistency.

For a malicious node v_{atk} , the attacker independently perturbs both the node features $\mathbf{x}_{v_{\text{atk}}}$ and the adjacency list. Therefore, the perturbation creates significant misalignment between the features and the structure, leading to a low consistency score. The probability that the consistency score $\phi(v_{\text{atk}})$ exceeds a pruning threshold δ is determined by the degree $d_{v_{\text{atk}}}$ of the node. Specifically, the probability decays exponentially with the degree: $\Pr[\phi(v_{\text{atk}}) \geq \delta] \leq \exp(-c \cdot d_{v_{\text{atk}}})$, where $c > 0$ is a constant depending on ϵ , and $d_{v_{\text{atk}}}$ is the degree of the malicious node. $\square$

D More Details on Experiments

D.1 Datasets

In this subsection, we provide a more detailed description of the four social network datasets used in our experiments (Table 3): LastFM, Twitch, Github, and Facebook. These datasets span diverse types of social networks, providing comprehensive benchmarks for evaluating graph learning methods under privacy constraints.

- **OS LastFM** [50] is a music-oriented social network where nodes represent users and edges represent friendships. Each node is associated with user features derived from music listening habits.
- **Twitch** [50] is a streaming platform network capturing user interactions and follow relationships. Node features encode information such as gaming preferences and activity patterns.
- **Github** [49] represents a developer collaboration network, where nodes correspond to users and edges indicate collaborations on repositories. Node features include user profile information and coding activity statistics.
- **f Facebook** [49] is a classic social network dataset where nodes represent users and edges denote friendships. Node features consist of demographic attributes and engagement metrics.

D.2 Effectiveness of Link Prediction

As shown in Fig. 8, the defense performance based on the link prediction is presented, validating the effectiveness of our approach.

E Related Works

E.0.1 Data Poisoning Attacks on GNNs. Data poisoning attacks aim to disrupt the training of machine learning models by injecting carefully crafted malicious data. In GNNs, such attacks are particularly effective because of the complex dependencies among nodes, edges, and overall graph topology that these models leverage for representation learning [54, 78–80]. Existing studies primarily consider conventional GNN settings with full access to node features and adjacency information. By contrast, attacks in locally private graph learning face stricter constraints: nodes and edges are perturbed locally, the attacker has limited knowledge of the true graph, and the protocol adds structural obfuscation. These conditions require novel strategies to meaningfully degrade model utility.

E.0.2 Locally Private Graph Learning. Locally private graph learning protocols generalize LDP to graph-structured data, enabling each user to perturb their sensitive features and adjacency information before uploading [15, 16, 20, 34, 38, 45, 47, 51, 77]. By leveraging multi-layer message passing in GNNs [43, 44], these protocols can tolerate noisy inputs and still produce useful embeddings for downstream tasks. However, the decentralized and obfuscated nature of locally private graph learning creates new attack surfaces, making them vulnerable to malicious injections that exploit both feature and structural perturbations. This underscores the need for defense strategies specifically designed for locally private graph learning.

E.0.3 Data Poisoning Attacks on LDP Protocols. Several prior studies [4, 19, 22, 33, 36, 58, 60, 68] have investigated data poisoning attacks in the context of Local Differential Privacy (LDP), primarily focusing on non-graph learning scenarios. These studies address various aspects of data poisoning, including key-value and frequency estimation [4, 58, 60, 68], mean and variance estimation [33], trajectory recovery [22], and range query analysis [36]. These works highlight the vulnerability of LDP protocols to malicious data injections, which can degrade the accuracy of estimates and compromise privacy guarantees. Recent research [16] has also started to examine data poisoning attacks on locally private graph learning protocols. However, this study primarily focuses on partial-privacy settings, and effective defense mechanisms are still lacking. This paper aims to extend the analysis by evaluating attacks under a broader range of privacy configurations and proposing more robust defense strategies for graph-based LDP protocols.